



Harness 잘 사용하기

챗봇과 싸우지 않고, 에이전트가 일할 환경을 설계하는 방법

게임플랫폼 1팀 라승수

시작하며

01

챗봇과 말씨름

맥락 충돌과 반복 설명

02

도구들의 수렵

자동완성·채팅 UI·터미널 에이전트

03

Harness

규칙·컨텍스트·권한·검증·상태 기록

개발자의 핵심 역량: AI에게 할 말 → AI가 안전하게 일할 환경

OPENING

전체 목차

01	코딩은 어디로 가고 있는가	개발자의 숙련 이동
02	왜 Claude Code인가	AI 코딩 도구의 구조 수렴
03	AI 시대의 개발 방법론	TDD·SDD·Spec-first 재부상
04	Harness	Prompt 이후의 에이전트 운영 구조
05	한계와 실패 패턴	긴 작업·보안·신뢰 문제
06	멀티 에이전트	역할과 컨텍스트 분리
07	워크플로우와 도구	명령어·게이트·워크트리·이슈
08	제작 과정	발표 제작 파이프라인 해부
09	다음에 해야 할 일	개인과 팀의 첫 변화

CHAPTER 01

코딩은 사라지는가

기계어 → 어셈블리

기계어

```
10111000 00000001 00000000 00000000 00000000
```

어셈블리

```
MOV EAX, 1
```



컴파일러가 만든 코드가 사람보다 효율적일 리 없다!

어셈블리 → C/Pascal

어셈블리

```
section .data
    msg db "Hello World"

section .text
    global _start

_start:
    mov rax, 1
    mov rdi, 1
    mov rsi, msg
    mov rdx, 12
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```



C/Pascal

```
#include <stdio.h>

int main() {
    printf("Hello\n");
    return 0;
}
```

진짜 프로그래머는 파스칼 같은 걸 쓰지 않는다!

C → Java

C

```
#include <stdio.h>
#include <stdlib.h>

...

int main() {
    Point* p = (Point*)malloc(sizeof(Point));
    p->x = 10;
    p->y = 20;
    printf(
        "Point: %d, %d\n",
        p->x,
        p->y
    );
    free(p);
    return 0;
}
```



Java

```
Point p = new Point(10, 20);
System.out.println(
    "Point: " + p.x + ", " + p.y
);
p = null;
```

메모리를 직접 관리하지 않으면 진짜 개발자가 아니다!

Java → Python

Java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```



Python

```
print("Hello, World!")
```

장난감 언어로 무슨 개발을 하냐!

AI 개발

자연어

이 함수를 리팩토링하고 테스트 코드를 작성해줘.

자연어로 시키는 건 진짜 개발이 아니다!

직접 하던 일의 감소

시대	전환	위임한 것
1950s	기계어 → 어셈블리	이진 코드를 직접 작성하는 부담
1960s	어셈블리 → C	레지스터와 명령어 최적화를 컴파일러에 위임
1990s	C/C++ → Java	메모리 해제를 가비지 컬렉터에 위임
2000s	언어 → 프레임워크	반복되는 보일러플레이트를 프레임워크에 위임
2010s	온프레미스 → 클라우드	서버 운영과 확장을 클라우드에 위임
2020s	직접 코딩 → AI 에이전트	코드 작성의 상당 부분과 작업 환경 설계

직접하는 일을 줄고, 설계하는 일은 늘어난다

숫자로 보는 변화

25%

YC W25

코드베이스 95% 이상 AI

100만 줄

OpenAI Codex 팀

수동 코드 한 줄 없음

3개월+

Meta 엔지니어

직접 코드 타이핑 없음

문서가 코드

이름은 다르지만 역할은 동일



Claude Code

CLAUDE.md



Cursor

Cursor Rules



GitHub Copilot

**copilot-
instructions.md**



Codex

AGENTS.md

개발자의 새로운 역할

코드를 잘 짜는 능력



문서를 잘 쓰는 능력



컨텍스트를 설계하는 능력

건축가

벽돌 쌓기 대신 설계도 작성

정확한 설계도 → 튼튼한 건물

오케스트라 지휘자

직접 연주 대신 전체 조율

CLAUDE.md = 총보

그래도 기초가 중요하다

AI가 더 많이 해줄수록 기초 지식을 가진 사람의 경쟁력 상승

변하는 것

직접 코드를 타이핑하는 비중

줄어드는 것은 타이핑의 비중

변하지 않는 것

시스템을 이해하는 능력

안전성 · 유지보수성 · 기존 시스템 적합성

인증과 권한 · 동시성 · 데이터베이스 스키마 · 캐싱

CHAPTER 02

왜 Claude Code인가

에이전틱 코딩의 실제 성과

2주

CTO 추정 4~8개월

프로젝트 완료

카카오 AI 팀 내부 공유 사례

1~2일

신규 개발자 온보딩

기존 수주~수개월에서 단축

첫날부터 의미 있는 기여

**불가능하던 작업
실현**

기술 부채 해결

즉각 피드백과 작업 실현

문서화된 작업 환경

1막: Copilot과 ChatGPT, 프롬프트의 시대

2022~2024

GitHub Copilot

현재 파일 기반 다음 줄 제안

현재 파일

암묵적 프롬프트

자동완성



ChatGPT

자연어 지시 기반 코드 생성

프롬프트 길이

역할 부여

단계별 지시

자연어 지시문이 새로운 프로그래밍 인터페이스처럼 보이던 시기

CoT / ReAct / ToT

Chain-of-Thought

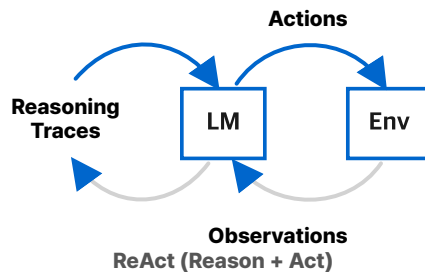
CoT



중간 추론 단계

ReAct

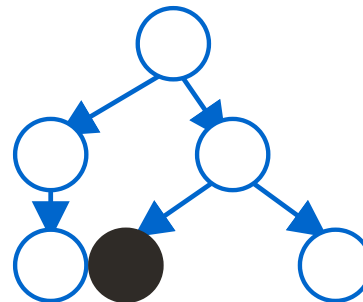
Reason + Act



추론과 행동 반복

Tree-of-Thought

ToT



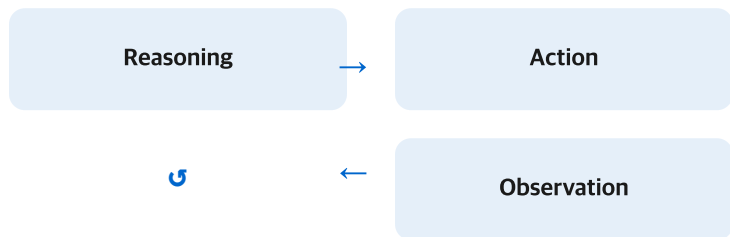
여러 추론 경로

ReAct / Tree-of-Thought

02

ReAct

추론과 행동을 번갈아 수행

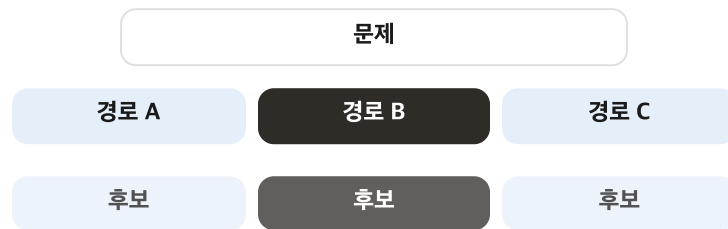


검색과 도구 호출이 루프 안으로 진입

03

Tree-of-Thought

여러 추론 경로를 동시에 탐색



하나의 답이 아니라 후보 경로를 비교

Self-Refine / Reflexion

Self-Refine

자기 출력을 다시 비판하고 수정

자기 출력 → 비판 → 수정 → 피드백 루프

출력 이후의 피드백 루프

Reflexion

자기 출력을 다시 비판하고 수정

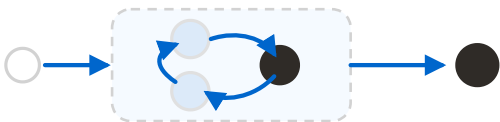
자기 출력 → 비판 → 수정 → 피드백 루프

출력 이후의 피드백 루프

네 가지 에이전틱 패턴

Reflection

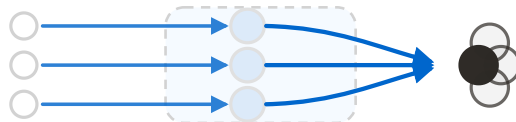
자기 결과를 비판하고 반복 개선



자기 수정 루프

Tool Use

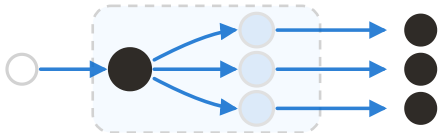
외부 도구로 정보 수집과 실행



모델 바깥 세계 연결

Planning

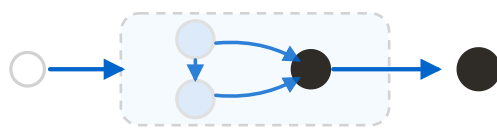
목표를 실행 순서로 나눠 추진



긴 작업 추진

Multi-Agent Collaboration

전문 역할로 나눠 복잡한 작업 수행



생성과 검증 분리

프롬프트 시대의 벽

관련 파일 부재

기존 유틸리티 인식 불가

기존 설계 부재

설계 결정 활용 불가

팀 컨벤션 부재

내부 규칙 활용 불가

모델은 컨텍스트 창 안에 들어온 지식만 다룰 수 있다.

Vivek Trivedy, LangChain

2막: Cursor와 컨텍스트의 시대

초기 Copilot

현재 파일 중심
검색 없음 또는 제한
한 줄 또는 한 블록



Cursor 계열 도구

전체 코드베이스
RAG · AST · 시맨틱 검색
멀티파일 수정과 Tools

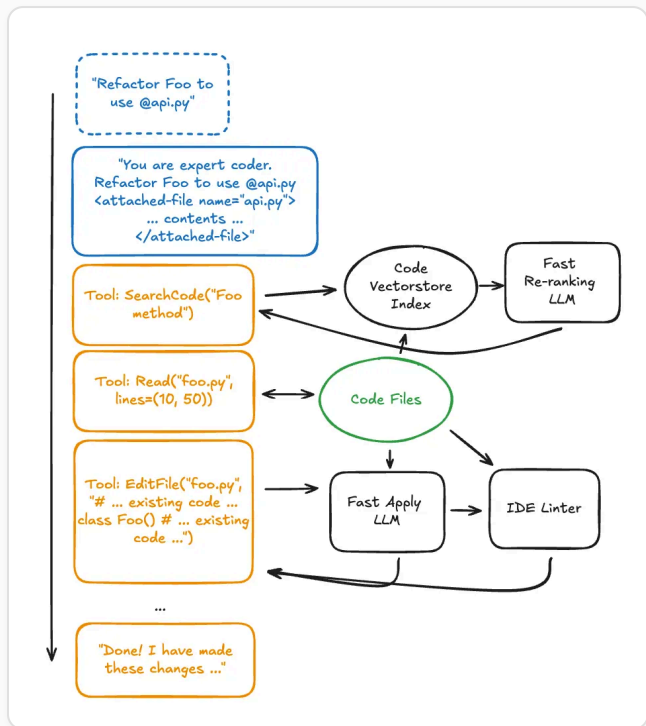
채팅

Composer

Agent Mode

Context Engineering

Cursor 아키텍처



컨텍스트 시대의 벽

컨텍스트 시대의 질문

무엇을 보여줄
것인가

좋은 입력만으로는 루프를
통제할 수 없다

필요해진 통제

무엇을 하게 할 것인가

무엇을 못 하게 막을 것인가

어디서 멈출 것인가

어떻게 검증할 것인가

도구 호출 실패

테스트 오해

비용 폭주

위험 명령

보안 경계

목표 망각

3막: 하네스의 시대

코딩 도구는 이제 실행 환경을
품는다

자동완성·채팅



작업 환경 전체
파일 · 셀 · 테스트



Harness

시대의 흐름

2022~2024

Prompt Engineering

어떤 말을 해야 하나

2025 중반~

Context Engineering

어떤 정보를 넣어야 하나

2026~

Harness Engineering

어떤 시스템을 만들어야 하나

Agent = Model + Harness

Prompt $\subset$ Context $\subset$ Harness

CHAPTER 03

AI 시대의 개발 방법론

왜 지금 방법론

시기	현상	대표 키워드/도구	핵심 질문
2025.02	AI 코딩의 대중화와 문화적 전환점	Vibe Coding	일단 빨리 만들어 보자
2025 중반	생성 코드의 품질, 보안, 유지보수 문제 부각	구조화된 검토, 검증 루프	이 결과를 믿어도 되나?
2025 하반기	구현 전에 방향을 고정하려는 업계 대응	SDD, Spec-first, GitHub Spec Kit	애초에 무엇을 만들라고 할 것인가?
2026 초	구현 뒤 검증 체계와 문맥 통제가 다시 중요해짐	TDD, Context Engineering	정말 그 스펙대로 동작하는가?

AI에게 무엇을 시킬지, 어떻게 검증할 것인지

SDD

Spec-Driven Development

SPECIFY

/speckit.specify

WHAT / WHY 분리

[NEEDS CLARIFICATION]

PLAN

/speckit.plan

HOW 제안

Constitution Check

TASKS

/speckit.tasks

태스크 분해

병렬 가능 작업 [P] 마킹

강제되는 원칙

WHAT/WHY와 HOW를 분리하고, 모호한 부분은 [NEEDS CLARIFICATION]에서 멈춘다.

핵심

모든 산출물이 파일로 저장되고 커밋되고 계속 참조된다.



GitHub Copilot icon GitHub spec-kit

TDD (Test-Driven Development)

테스트를 먼저 쓰고, 통과하는 코드를 나중에 쓴다. AI 시대에는 인간이 테스트, AI가 구현.

Red

인간이 실패 테스트 작성

Green

AI가 통과 코드 구현

Refactor

인간 리뷰 → AI가 리팩토링

왜 AI에게 특히 중요한가

AI는 "동작하는 것 같은" 코드를 자신 있게 만들. 테스트 없으면 맞는지 틀린지 확인 불가.

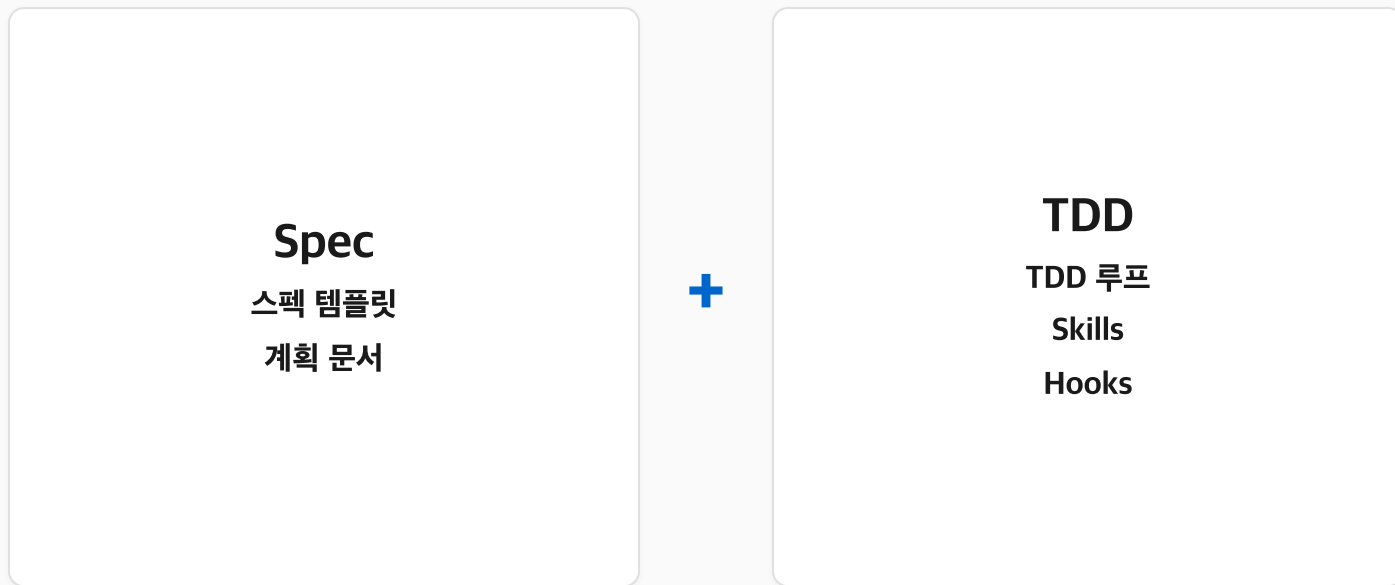
AI의 치팅에 주의

테스트 수정 금지 규칙 필수. 테스트 코드 임의 수정 금지 · assert 조건 약화 · 실패 확인 전 구현 금지.

Waterfall vs SDD

	Waterfall	SDD
개발 흐름	요구사항 → 설계 → 코딩 → 테스트	스펙이 primary artifact
검증 시점	테스트가 개발 후반에 실제 제약을 드러냄	/speckit.specify → /speckit.plan → /speckit.tasks
실행 산출물	요구사항 또는 설계를 다시 고침	spec · plan · tasks를 실행 산출물로 갱신

SDD + TDD가 Harness로 이어지는 이유



이 시스템이 곧 하네스 엔지니어링

CHAPTER 04

프롬프트를 넘어서

Prompt / Context / Harness

Prompt, Context, Harness

층위	핵심 질문	초점
Prompt	무엇을/어떻게 말할 것인가	명령의 품질 / 단일 프롬프트 / 정적
Context	무엇을/어떻게 보여줄 것인가	정보의 품질 / 입력 파이프라인 / 동적
Harness	무엇을/어떻게 통제할 것인가	시스템의 안정성 / 실행 환경 전체

Prompt $\subset$ Context $\subset$ Harness

Agent = Model + Harness

모델이 아닌 것은 전부 하네스입니다.

LangChain, Vivek Trivedy

Context Engineering

모델이 봐야 할 정보와 버려야 할 정보

Tool Orchestration

도구 호출 순서와 권한

State & Memory

세션을 넘어 남는 파일과 기록

Verification Loop

테스트, 린트, 리뷰

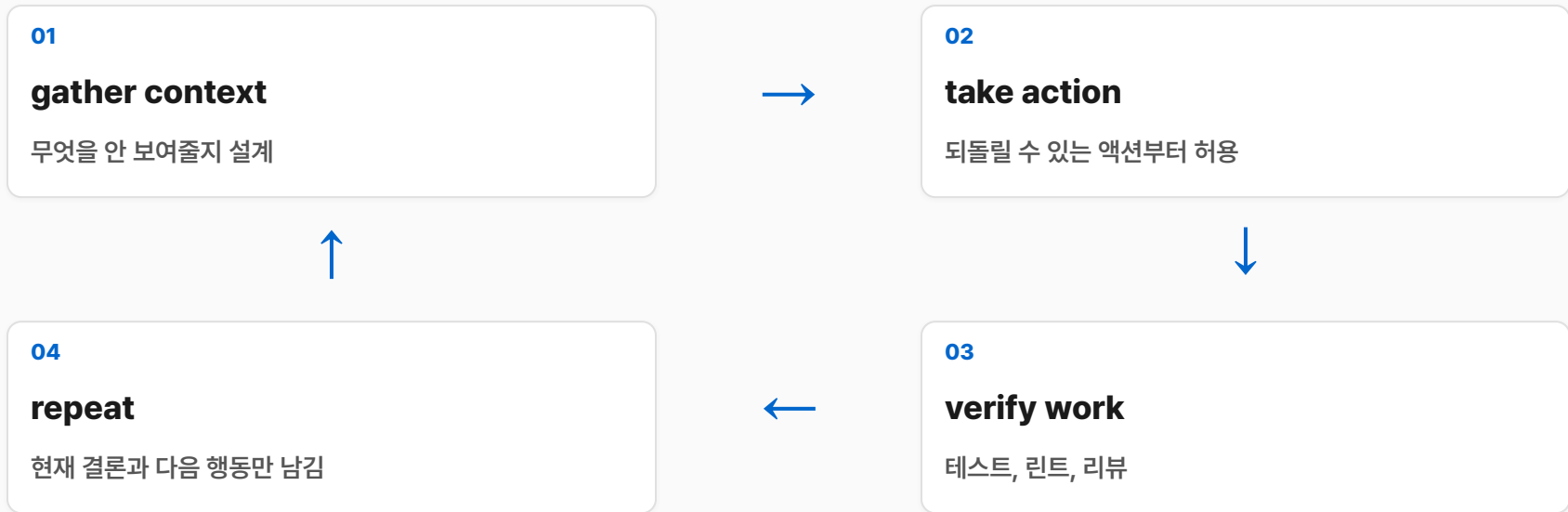
Error Recovery

실패 감지와 재시도, 우회, 중단

Human-in-the-Loop Control

사람 승인과 개입 지점

에이전트 루프: 하네스의 심장



거의 모든 에이전트가 반복하는 4단계.

하네스의 책임

01

Guardrails

위험한 행동과 권한 경계

02

Specification

작업 범위와 기준선

03

Verification

결정론적 결과 검증

04

State Management

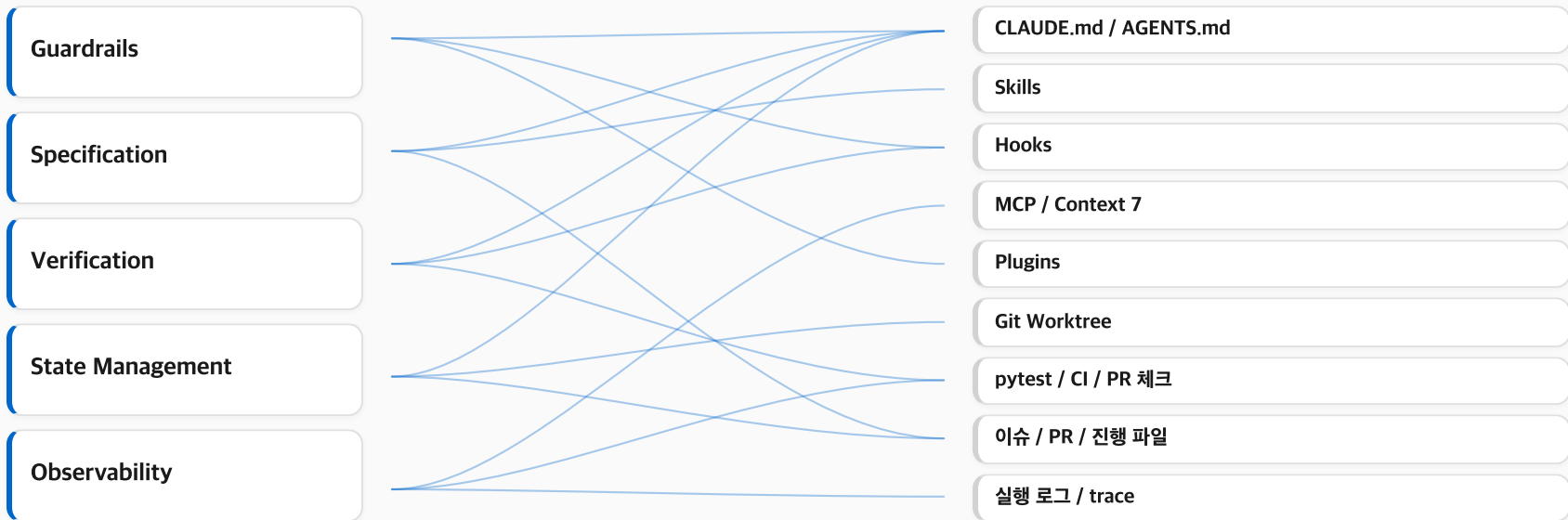
파일, Git 이력, 진행 기록

05

Observability

실행 로그, trace, checkpoint

하네스의 도구



책임과 도구는 1:1이 아니다

Context Engineering

01

Write

역할, 목표, 금지, 출력, 검증
기준을 구조화

02

Select

지금 필요한 문서와 파일만 선택

03

Compress

결정, 남은 일, 증거 중심으로
압축

04

Isolate

대량 탐색과 로그 분석은 격리

Anthropic의 4가지 전략: 필요한 정보만 남기고 잡음은 덜어낸다.

Anthropic Research

MCP와 Context 7

에이전트가 MCP를 쓰는 구조



도구 결과가 컨텍스트 윈도우로 돌아와 다음 판단의 재료가 됨

GitHub · Slack · DB · Filesystem · Internal API

Context 7 MCP

최신 문서를 읽힌 뒤 답하게 하는 MCP 서버

역할	최신 API 문서와 공식 자료를 필요할 때 가져옴
원리	모델 기억 대신 지금 기준 문서를 읽게 함
효과	SDK/API처럼 바뀌는 문서에 강함

RAG vs Context 7

검색 범위와 기준 시점의 차이

구분	RAG	Context 7
대상	문서 저장소에서 검색된 관련 조각	지정한 최신 문서나 공식 자료
강점	내부 위키, 정책 문서, 긴 참고 자료를 넓게 훑음	최신 SDK, API, 변경이 잦은 공식 문서를 지금 기준으로 확인
기대	넓은 문서 집합에서 필요한 맥락을 찾음	출처와 기준 시점이 더 분명함
문제점	오래된 조각, 낮은 관련도, 중복 정보가 섞일 수 있음	연결 실패나 잘못된 소스 지정에 바로 흔들림

Memory: 세션을 넘어서는 기억

CLAUDE.md / AGENTS.md

규칙 파일과 기준선

프로젝트 노트와 결정 기록

설계 의도와 합의

이슈와 PR 히스토리

작업 맥락과 리뷰 흔적

progress.txt / prd.json

진행 상태와 요구사항

벡터 DB / 코드 그래프

검색 가능한 프로젝트 지식

커밋 히스토리와 아키텍처 결정 기록

변경 이유와 구조 결정

대화창을 기억 저장소로 착각하지 않는다

Stable Prefix와 Variable Suffix

Stable Prefix

KV-cache / 고정 / 재사용

시스템 프롬프트

도구 정의

장기 규칙

앞쪽의 안정 접두어



Variable Suffix

동적 접미어 / 갈아 끼움

최신 사용자 입력

도구 결과

뒤에서 갈아 끼움

최신 입력과 도구 결과

최신 입력과 도구 결과만 뒤에서 갈아 끼워야 KV-cache hit rate가 살아납니다.

Manus Research

하네스는 환경 그 자체다

01

필요한 파일

상태와 근거를 세션 밖에 남김

02

필요한 도구

에이전트가 실제 일을 할 수 있게 연결

03

필요한 규칙

권한, 검증, 중단 지점을 고정

Harness Builder는 에이전트가 쓸 수 있는 환경을 만드는 사람

CHAPTER 05

이렇게 하면 망한다

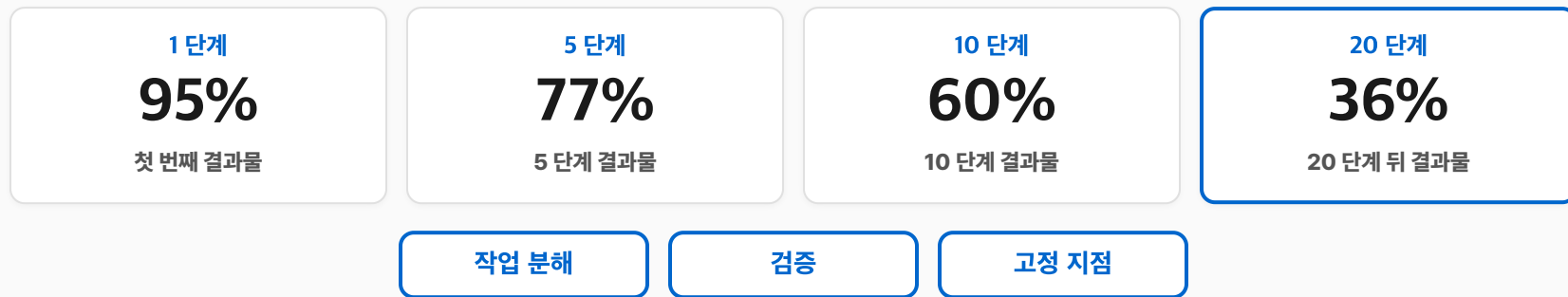
오류의 나비효과

계획 > 탐색 > 작성 > 테스트 > 수정 > 커밋

단계 수	전체 성공률
1 단계	95%
5 단계	77%
10 단계	60%
20 단계	36%
50 단계	8%

20 단계만 가도 2/3이 실패

작업이 길어질 때 특히 위험한 이유



컨텍스트가 길수록 항상 좋은 것은 아니다

지금 판단에 필요한 신호만 남기는 것이 능력

관련 정보가 있음 ≠ 제때, 제 위치, 올바른 기준

오래된 가정

실패한 도구 결과

폐기된 계획

불필요한 파일

신호 대 잡음비

대표 실패 패턴 네 가지

POISONING

오래된 가정 / 잘못된 메모 / 판단 오염

DISTRACTION

정보 과잉 / 핵심 신호 매몰

CONFUSION

과거 결정과 현재 결정 / 최신 기준 혼선

CLASH

서로 다른 지시와 기준 / 한 컨텍스트 안의 충돌

기억과 잡음과 규칙이 한 컨텍스트 안에서 섞임

현실에서 보이는 증상들

AI Slop

산출물 조잡화

Doom Loop

같은 실수 반복

Shadow Agent

작업 이유와 기준 상실

모델 이상이 아니라 구조 문제

Context Rot: 길어진 기억은 조용히 썩는다

길이 문제가 아니라 흐려지는 의도와 제약

대화창 안의 상태

초반 규칙 약화

가정의 사실화

오래된 도구 결과 잔존

끊을 타이밍

현재 결론

다음 행동

바깥의 아티팩트

파일

이슈

진행 기록

테스트 결과

Git 이력

신뢰는 조율되어야 한다

신뢰는 감정이 아니라 운영 문제

넓은 자율성

초안 작성

반복 리팩터링

구조 탐색

강한 검증

배포

삭제

민감 데이터

최신 API

외부 인용

보안 코드

똑똑함 ≠ 그대로 믿어도 됨

결정 제어와 확률 제어를 분리하라

기계가 확인할 것은 기계에게

결정론적 제어

린트

타입 체크

테스트

포맷

위험 명령 차단

파일 권한

승인 게이트

AI는 판단과 생성에 집중

확률적 제어

설계 대안 제안

코드 리뷰 의견

아키텍처 판단

요약

우선순위 평가

더 긴 컨텍스트보다 더 좋은 게이트

누적되는 루프에는 통제 구조가 필요

조용히 커지는 것

작은 오류는 조용히 커짐

앞쪽 게이트

기계가 확인할 것은 앞쪽 게이트로

AI의 자리

AI는 판단과 생성에 집중

왜 하나의 에이전트만으로는 부족한가

01

컨텍스트의 벽

탐색·구현·로그 분석·리뷰·문서 작성이 같은 창에 쌓임

무관한 정보가 판단을 오염

02

역할의 벽

계획·구현·리뷰·테스트 해석이 한 손에 집중

자기 답을 자기가 채점

03

신뢰성의 벽

탐색만 해야 하는 작업에 쓰기 권한까지 열림

민감 데이터를 볼 필요가 없는 작업까지 전체 컨텍스트 수신

멀티 에이전트는 이 문제를 줄이기 위한 구조

하나의 에이전트 = 하나의 역할

01

Sub-Agent

Main → Sub

중간 작업을 격리하는 기본형

단일 심부름

02

Orchestrator

Plan → Exec → Exec

계획자 하나가 여러 실행자를 배치

1:N 위임

03

Parallel

A → shared repo → B

같은 목표를 평면으로 벌림

독립 lane

04

GAN-Style

Gen → Eval

생성과 평가를 분리

루브릭으로 평가

05

Agent Teams

R → C → E

양방향 대화와 합의

그래프 구조

Planner / Generator / Evaluator: 생성과 평가를 같은 손에 쥐지 않음

메인 컨텍스트에 무엇을 남기고, 무엇을 밖으로 밀어낼 것인가

1. Sub-Agent: 중간 작업을 격리하는 기본형



컨텍스트 격리

파일을 많이 읽고 수만 토큰을 써도 메인 컨텍스트는 오염되지 않음

least privilege

필요한 파일, 질문, 출력 형식, 권한만 전달

좋은 지시

목적, 읽을 범위, 출력 형식, 근거 표시 방식, 금지 행동

Sub-Agent는 메인 컨텍스트를 지키는 데 특히 강함

Advisor 전략: 작은 실행자, 큰 자문

executor

규모가 작은 저렴한 모델이 실제 일을 계속 맡음

메인 실행자는 그대로 유지

advisor

막히는 순간에만 더 상위 모델에게 판단을 빌림

다음 계획 · 방향 수정 · 여기서 멈춰라

복잡한 아키텍처 분기, 막힌 디버깅, 위험한 계획 수정에 필요한 일시적 판단

2. Orchestrator: 계획자 하나가 여러 실행자를 배치한다

Planner

목표 해석 · 하위 작업 분해 · 실행자 배정

Exec

하위 작업

Exec

하위 작업

Exec

하위 작업

결과 수집 → 전체 통합 → 재작업 지시

품질은 분해의 품질과 수렴 지점의 품질

3. Parallel: 같은 목표를 평면으로 벌리고 나중에 합치기

A

대안 비교

세 가지 라이브러리 대안

B

설계안 생성

같은 요구사항에 여러 설계안

C

독립 점검

서로 다른 파일 묶음

Git Worktree · 별도 브랜치 · 별도 작업 디렉터리 · 출력 계약

좋은 병렬은 서로를 더럽히지 않는 구조

4. GAN-Style: 생성자와 평가자를 분리한다

Planner

제품 스펙과 작업 범위

사용자 요청 확장

Generator

기능 또는 스프린트

한 번에 하나씩 구현

Evaluator

테스트·브라우저·루브릭·리뷰

별도 관점으로 평가

구체적인 피드백을 다시 Generator에게 돌려보냄

생성과 평가는 같은 손에 묶지 않음

5. Agent Teams: 양방향 대화가 가능한 팀을 만든다

Researcher

외부 문서와 사례

Architect

설계 대안

Reviewer

위험과 누락

Coordinator

논의를 수렴

Editor

최종 품질

양방향 대화

여러 독립 세션이 서로 대화하며 수렴

복수 관점

연구, 전략 문서, 대규모 아키텍처 검토, 제품 기획

보통 3명 안팎에서는 이득, 너무 커지면 AI 회의만 길어질 수 있음

Sub-Agent와 Agent Team은 다르다

Sub-Agent

메인이 작업을 던지고 결과를 받음 / 제한 범위, 요약 반환

단방향 통신 / 로그 파싱, 파일 탐색, 단일 검증 / 낮은 요약 품질

Agent Team

서로 대화하며 수렴 / 별도 컨텍스트와 공유 맥락

양방향 통신 / 관점 충돌, 기획, 아키텍처 토론 / 수렴 실패

양방향 토론이 필요한 경우에만 Agent Team

설계 원칙: 패턴보다 경계가 중요하다

01

필요한 것만 받기

컨텍스트를 많이 주는 것은 친절이 아님

02

역할과 금지 행동

무엇을 하고 무엇을 안 하는지 먼저 결정

03

출력 계약 구조화

완료 기준과 반환 형식을 먼저 고정

04

생성자와 검증자 분리

자기 결과를 자기가 통과시키지 않기

05

중단 조건과 수렴 조건

몇 번 재시도할지, 언제 넘길지 미리 정하기

패턴 이름보다 중요한 것은 경계 설계

설계 원칙: 패턴보다 경계가 중요하다

S

단일 책임

코드: 클래스를 분리한다

에이전트: 역할과 도구를 분리한다

O

개방-폐쇄

코드: 수정 없이 확장한다

에이전트: 훅과 플러그인으로 확장한다

I

인터페이스 분리

코드: 불필요한 메서드 의존을 줄인다

에이전트: 불필요한 도구와 파일 권한을 노출하지 않는다

D

의존성 역전

코드: 추상화에 의존한다

에이전트: 하드코딩 프롬프트 대신 컨텍스트와 규칙을 외부화한다

멀티 모델과 멀티 에이전트

Claude

구현 계획

긴 맥락과 구현 루프 중심

Codex

구조 검토

코드 구조·리팩터링·코드 리뷰

Gemini

가독성 피드백

UI·문서·대안적 시각

합의 · 불일치 · 불확실 분리

결론: 더 많은 AI가 아니라 더 좁은 역할의 AI 여럿

01

메인 컨텍스트

판단에 필요한 신호만 남김

02

판단 책임

계획·실행·리뷰를 나눔

03

검증 관점

생성 결과를 다른 기준으로 판정

더 좁은 역할의 AI 여럿을, 더 명확한 경계 안에서 일하게 한다

시작하며: 두 가지 막다른 길

모든 것을 직접 세팅

명령어를 매번 손으로 조합

검증 루프를 기억에 의존

세션마다 비슷한 지시를 다시 입력

원리 없이 Skills·플러그인·프리셋

이상 동작이 나오면 원인을 읽지 못함

시간이 지나면 팀에 블랙박스만 남음

원리가 아니라 도구만 축적

어떤 원리를 시스템이 대신 강제하게 만들 것인가

OMC(Oh My Claude Code)

01

역할 분리된 여러 에이전트

작업 성격에 맞는 역할

/model

모델 라우팅

모델 선택

/ralplan

Plan-Critic-Build 명령

계획과 비평 분리

/ultrapilot

병렬 worker

worker를 띄움

/ultraqa

QA와 검증 명령

test → fix → 재실행 루프

06

Git Worktree 기반 격리

이슈와 PR 연동

작업의 성격만 말하면 시스템이 모드, 역할, 모델, 검증 순서를 먼저 잡음

Plan-Critic-Build

Plan → Critic → Build

Explore → Plan → Code → Commit

success criteria

필요없는 도구는 덜어내라

01

tool curation

필요한 도구만 남김

02

lint / type check

기계가 달는 문

03

테스트 / 리뷰 / 승인

검증과 승인

04

PreToolUse / PostToolUse

위험 명령 차단, 배포 전 human-in-the-loop

기계가 확인할 수 있는 것은 기계가 닫아야 함

Approval, Auto-accept, Plan Mode

01

문서 초안, 로그 요약

비교적 넓은 자율성

실패 비용이 낮고 되돌리기 쉬움

02

테스트 보강, 작은 리팩터링

제한적 auto-accept + 검증

반복 작업이지만 검증 필요

03

아키텍처 변경, DB 마이그레이션

Plan Mode + 승인

잘못되면 비용이 큼

04

삭제, 배포, 권한 변경

수동 승인 필수

상태 변경과 사고 위험이 큼

작업별로 신뢰 수준을 조절하는 구조

반복의 자산화

`/context`

지금 무엇이 얼마나 쌓였는지

컨텍스트 확인

`/compact`

이어갈 결정과 상태만

압축

`/clear`

다른 작업으로 넘어갈 때

세션 비움

Skills

반복 지시

Hooks

우회하면 안 되는 규칙

Custom Commands

자주 쓰는 루프

CLAUDE.md / AGENTS.md

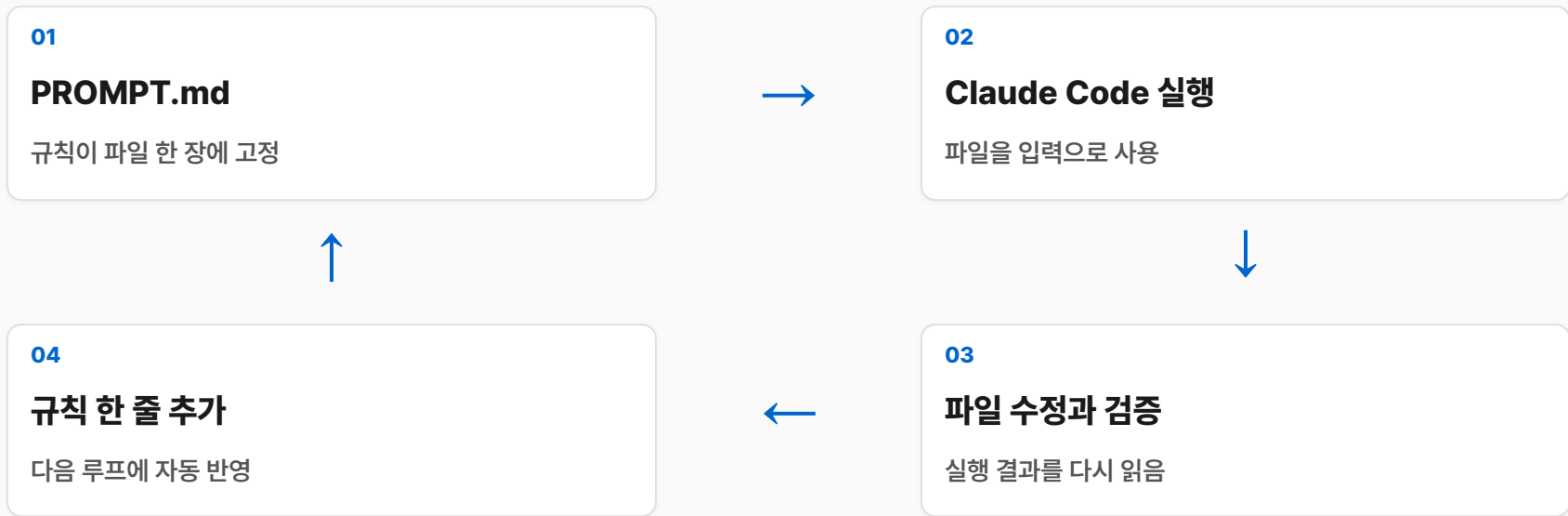
프로젝트 공통 기준

이슈나 진행 파일

특정 태스크의 상태

개인 메모에서 리포지토리와 팀 설정으로 올라가면 표준이 됩니다

Ralph Loop



단순성은 예측 가능성

암묵지를 파일로 뽑아내라

01

결정의 이유

결과만 남기지 않음

02

Post-Mortem → 규칙

갈라진 지점을 한 줄로 승화

03

피드백마다 규칙 성장

반복 피드백은 파일로 이동

04

취향 encode

전용 에이전트나 규칙 파일

머릿속 판단이 파일로 옮겨지는 순간 하네스가 됩니다

한 모델에만 기대지 않는다

01

Claude

긴 맥락과 구현 루프 중심

02

Codex

코드 구조, 리팩터링, 코드 리뷰 및 검토

03

Gemini

UI, 가독성, 대안적 시각, 문서 피드백

합의



불일치



불확실

합의하면 진행, 불일치하면 사람 검토, 불확실하면 추가 자료

cmux와 Git Worktree

창 1

메인 구현 세션

결정과 통합

창 2

조사와 탐색 전용

코드베이스 탐색과 문서 검색

창 3

테스트와 감시 로그

테스트·빌드 로그 확인

창 4

이슈와 PR 정리

이슈 확인, PR 생성

Git Worktree는 이 분리를 파일 시스템 수준에서 고정

세션이 아니라 이슈가 상태를 들고 간다

이슈

세션보다 오래가는 상태 저장소

01

작업 목적과 수용 기준

Linear/PMS/GitHub Issue

02

Worktree 생성

이슈에서 시작

03

에이전트 세션

해당 이슈 범위만

04

작은 커밋

변경 단위 기록

05

이슈와 PR에 증거 첨부

검증 결과 연결

06

완료 판단

수용 기준과 검증 증거

이슈 → Worktree → 세션 → PR → 검증

첫 주에 바로 세울 수 있는 최소 워크플로우

01

CLAUDE.md / AGENTS.md

반복 지시 세 가지

02

위험 명령 차단

혹이나 승인 규칙

03

lint / type check

필수 게이트 하나

04

이슈 하나와 Worktree 하나

긴 작업 묶기

05

계획과 승인

구현 전 계획, 승인 후 수정

06

결정과 남은 일

파일이나 이슈에 남김

대화창에 떠다니는 기준을 작업 환경의 일부로 만드는 첫 단계

결론: 실전 워크플로우의 중심은 운영 구조다

01

원리를 명령어로 고정

계획과 실행을 분리

02

결정론적 게이트

무엇을 어디서 검증할 것인가

03

상태와 기준의 흔적

외부 아티팩트에 남김

무엇을 어디서 검증하고, 어떤 흔적 위에 다음 판단을 올릴 것인가

CHAPTER 08

이 글과 발표가 만들어진 과정

이 글과 발표가 만들어진 과정

기억하시나요

이 발표를
코드 한 줄 없이 만든 방법

한 것은 하나 - 하네스 설계

지금부터 그 증거

이 발표를 만든 방법

01

source

PDF, HTML, 외부 링크,
로컬 마크다운 자료

02

prose

발표와 글의 기준 문장

03

outline

slide 단위와 장 구조

04

html

Spec, Harness, Skills

05

pdf

markdown, html,
규칙 문서

자료 1 source 수집

자료 수집

**유튜브 · 링크드인 · 인터넷
커뮤니티**

서로 다른 계층

자료 수집

공식 사이트 · 공개 문헌

서로 다른 중요도

자료 수집

직접 수집한 마크다운 · PDF

불확실한 부분을 억지로 메우지 않는 구분

재료 2 단일 진실원 만들기

01

prose

블로그 글처럼 작성

02

논리 순서

주장 강도를 책임지고 확인

03

source of truth

HTML slide, PDF 구성 기준

04

사람이 개입

한국어 문체와 독자 경험

자료 3- slides-grab, Skill 경계로 박힌 분리

PLAN

slide-outline.md

승인된 아웃라인

DESIGN

slide-XX.html

사용자 승인까지 반복

EXPORT

PDF 변환

산출물 검증

재료 4 규칙 세우기

DESIGN CONTRACT	VALIDATION
톤과 분위기	slide contract 점검
색상 팔레트	한국어 문장 점검
타이포그래피	deck-level consistency 확인
여백과 레이아웃 규칙	source와 주장 매핑 확인
금지해야 할 표현과 과한 장식	디자인 계약 위반 확인

MS slides · Stitch Design.md · Kuneosu/make-slide

생성과 검증을 같은 손에 쥐지 않았다

HTML generation

HTML generation

생성자가 자기 결과를 스스로 통과시키게 두면

그럴듯한 오류의 장기 생존



기계적 검증 + 사람 검토

기계적 검증

사람 검토

slide contract 점검

한국어 문장 점검

source와 주장 매핑 확인

디자인 계약 위반 확인

생성과 검증 분리

이 발표가 증거

01

원하는 것을

글로 작성

02

규칙을

문서로 작성

03

결과를

사람이 검증

CHAPTER 09

우리가 다음에 해야 할 일

시작하며: FOMO 와 피로

"지식이 쌓이는 속도보다 감가상각되는 속도가 더 빠르다. 나도 지쳤다."

Simon Willison, 2025

최전선의 피로

AI FoMO

무엇을 남기고 무엇을 버릴지

FOMO-AI • 2025

뒤처지는 두려움

뒤처질까 두려움

좋은 도구를 못 쓸까

남들만 이득 볼까

직접 써 보는 일

증폭되는 경험 - 세 개의 증언

세 사람이 거의 같은 말을 한다.

KENT BECK

TDD 창시자 · 52년

프로그래밍은 여전히 프로그래밍이고,
AI는 더 중요한 결정을 더 빨리 내리게
하는 도구다

SIMON WILLISON

Vibe Engineering

AI는 기존 전문성을 증폭한다. 경험이
많을수록 더 빠르고 더 좋은 결과를
얻는다

MARTIN FOWLER

Thoughtworks

AI는 주니어의 산출물은 복제할 수
있어도 시니어의 경험과 판단은
복제하지 못한다

AI 시대의 경험 - 축소가 아니라 증폭

Spec-first · TDD · Plan-Critic · 검증 분리

내일부터

글로 먼저

CLAUDE.md 1페이지 작성, 또는 반복 업무 1개를 글로 명세

직접 써보기

터미널 열고 Claude Code 3~4창 - 토큰을 일부러라도 소비

도구

Oh My Claude Code 설치 - 원리의 자동화

"토큰을 많이 쓰는 사람이 가장 경쟁력 있는 개발자" - 박종천